Bézier Curve — Matrix Method (Symbolic)

$P(u) = U \cdot M \cdot G$ via Symbolics.jl

Explanation of the Program Output

Case 2: Quartic 3-D, 5 Control Points

`05-bezier-matrix-sympy-text.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the output of `05-bezier-matrix-sympy-text.jl` when run with `CASE = 2`. This script is a hybrid: it uses the straight output structure of script 04 (five flat sections, no derivation steps) combined with exact `Rational{Int}` arithmetic throughout, and Symbolics.jl for polynomial formatting. The output is almost identical to script 04 with three specific differences.

Contents

1 What Changed from Script 04 **2**

2 Header **2**

3 G, M, and $M \cdot G$ — Unchanged in Appearance **2**

4 Resulting Polynomials — Symbolic Format **3**
4.1 Reading the Symbolics.jl notation 3

5 Evaluation Table — Exact u Values **4**
5.1 Exact u values 4
5.2 Exact result values 4

5.3 Decimal column 5

6 The Convergence of All Five Scripts **5**

1. What Changed from Script 04

Script 05 is script 04 with exact arithmetic substituted for floating-point. The output structure is identical — same five sections, same table format. The differences are surgical.

Three differences from script 04:

Location	Script 04	Script 05
u values in table	Decimals: `0.2500`, `0.5000`, `0.7500`	Exact fractions: `1//4`, `1//2`, `3//4`
Resulting Polynomials section	Plain strings: `13u^4 - 32u^3 + 24u^2`	Symbolics.jl format: `(24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)`
Section subtitle	`Resulting Polynomials`	`Resulting Polynomials (via Symbolics.jl)`

The G, M, and $M \cdot G$ matrices look identical because all their entries happen to be whole numbers. `Rational{Int}` displays integers without the `//1` suffix when the denominator is 1.

2. Header

```
==================================================================
  Bezier Curve - Matrix Method  [P(u) = U * M * G]  (Symbolics.jl)
  Case 2  |  Quartic  |  3D  |  5 Control Points
==================================================================
```

The title now includes `(Symbolics.jl)` to flag that the polynomial formatting uses the CAS. Everything else is unchanged from script 04.

3. G, M, and $M \cdot G$ — Unchanged in Appearance

```
  G  - Geometry Matrix (Control Points)
      x  y  z  ...  P0  0  0  1  ...  P4  5  4  1

  M  - Basis Matrix (derived via binomial)
       P0   P1   P2  P3  P4
  u^4   1   -4    6  -4   1  ...

  M * G  - Polynomial Coefficient Matrix
```

```
         x    y  z
 u^4    13  -28  0  ...
```

These three sections look byte-for-byte identical to script 04. The reason is that all values in G, M, and $M \cdot G$ are whole numbers. `Rational{Int}` stores them as fractions internally (e.g. `13//1`), but the formatting function `fmt_rat` checks whether the denominator is 1 and prints only the numerator in that case — so `13//1` displays as `13`, not `13//1`.

The subtitle change: `M -- Basis Matrix (derived via binomial)`

Script 04 said `derived from Bernstein`. Script 05 says `derived via binomial`. Both mean the same thing — the entries come from the binomial theorem applied to the Bernstein expansion — but script 05 is slightly more precise about the mechanism.

4. Resulting Polynomials — Symbolic Format

```
Resulting Polynomials  (via Symbolics.jl)
----------------------------------------------------------------

x(u)  =  (24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)
y(u)  =  (16//1)*u - (48//1)*(u^2) + (64//1)*(u^3) - (28//1)*(u^4)
z(u)  =  1
```

This section shows the most visible change from script 04. The polynomial strings are now produced by Symbolics.jl rather than by the hand-written `fmt_poly` function.

4.1. Reading the Symbolics.jl notation

The notation `(24//1)*(u^2)` has been explained in the script 02 output explanation. To recap:

- `24//1` is the `Rational{Int}` value $\frac{24}{1} = 24$, printed with the rational type made explicit.
- `*(u^2)` is Symbolics.jl's way of writing multiplication by u^2.
- So `(24//1)*(u^2)` means exactly $24u^2$.

Script 04 vs Script 05 for $x(u)$:

Script 04 (hand-formatted):

```
x(u)  =  13u^4  - 32u^3  + 24u^2
```

Script 05 (Symbolics.jl formatted):

```
x(u)  =  (24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)
```

Two things differ:

1. **Coefficient notation:** plain `13` vs `(13//1)`. Both mean the same number;

the rational wrapper is Symbolics.jl being explicit about the type.

2. **Term ordering:** script 04 prints from highest power down (u^4 first). Symbolics.jl prints from lowest power up (u^2 first) — this is its default canonical ordering.

The polynomials are mathematically identical.

Why does $z(u)$ still print as just `1`?

The z column of $M \cdot G$ is $[0, 0, 0, 0, 1]$ — the constant polynomial. When Symbolics.jl builds the symbolic expression, it gets $0 \cdot u^4 + 0 \cdot u^3 + \cdots + 1 \cdot u^0 = 1$. A constant has no variable, so there is no `(1//1)*u^0` — Symbolics.jl simplifies it to the plain integer `1`.

5. Evaluation Table — Exact u Values

```
Evaluation Table  -  P(u) = U * (M * G)
----------------------------------------------------------------


+==============================================================+
| u       x(u)                y(u)                z(u)         |
|==============================================================|
| 0       0  (0.0000)         0  (0.0000)    1  (1.0000)       |
| 1//4    269//256  (1.0508)  121//64 (1.8906) 1  (1.0000)     |
| 1//2    45//16  (2.8125)    9//4  (2.2500)   1  (1.0000)     |
| 3//4    1053//256 (4.1133)  201//64 (3.1406) 1  (1.0000)     |
| 1       5  (5.0000)         4  (4.0000)       1  (1.0000)    |
+==============================================================+
```

5.1. Exact u values

Script 04 showed `0.2500`, `0.5000`, `0.7500` in the u column. Script 05 shows `1//4`, `1//2`, `3//4`.

This is because `u_values` is now a `Vector{Rational{Int}}`, not a `Vector{Float64}`. The values $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$ are stored as exact fractions, and `fmt_rat` displays them as `1//4`, `1//2`, `3//4`.

5.2. Exact result values

The result values (`269//256`, `45//16`, etc.) are the same fractions as in script 02. The key difference from script 04 is *how* they are computed:

Script	How exact fractions are produced
Script 04	Compute with `Float64`, then call `rationalize()` to convert back to the nearest fraction. The fraction is recovered, not computed.
Script 05	Compute with `Rational{Int}` throughout. The fraction `269//256` is the direct result of integer arithmetic — never a float.

Both scripts display the same fractions. Script 05 arrives at them by a more rigorous route.

Verify $x\left(\frac{1}{4}\right) = \dfrac{269}{256}$ using `Rational{Int}` arithmetic:

The U vector at $u = \frac{1}{4}$ is:

$$U = \left[\left(\tfrac{1}{4}\right)^4,\ \left(\tfrac{1}{4}\right)^3,\ \left(\tfrac{1}{4}\right)^2,\ \tfrac{1}{4},\ 1\right] = \left[\tfrac{1}{256},\ \tfrac{1}{64},\ \tfrac{1}{16},\ \tfrac{1}{4},\ 1\right]$$

The x column of $M \cdot G$ is $[13, -32, 24, 0, 0]$.

Dot product (all as exact fractions):

$$\tfrac{1}{256}(13) + \tfrac{1}{64}(-32) + \tfrac{1}{16}(24) + \tfrac{1}{4}(0) + 1(0)$$

$$= \tfrac{13}{256} - \tfrac{128}{256} + \tfrac{384}{256} = \tfrac{13-128+384}{256} = \tfrac{269}{256}$$

No floating-point involved at any step. ✓

5.3. Decimal column

The decimal column (`1.0508`, `2.8125`, etc.) is produced by converting the exact `Rational{Int}` result to `Float64` at the very last moment: `Float64(269//256) = 1.05078125`, displayed as `1.0508` with 4 decimal places. The decimal is a presentation convenience, not the actual stored value.

6. The Convergence of All Five Scripts

Five scripts. Three methods. Two arithmetic systems. One curve.

Script	Method	Arithmetic	Polynomial display
01	Bernstein expansion	`Float64`	Hand-formatted
02	Bernstein expansion	`Rational{Int}`	Symbolics.jl
03	Matrix (pedagogical)	`Float64`	Hand-formatted
04	Matrix (straight)	`Float64`	Hand-formatted
05	Matrix (straight)	`Rational{Int}`	Symbolics.jl

Every script produces the same five curve points: $(0, 0, 1)$, $(1.051, 1.891, 1)$, $(2.813, 2.250, 1)$, $(4.113, 3.141, 1)$, $(5, 4, 1)$.

The method (Bernstein vs matrix), the arithmetic (`Float64` vs `Rational{Int}`), and the display format (hand-formatted vs Symbolics.jl) are independent choices that can be mixed freely. Script 05 completes the matrix side of the `Rational{Int}` + Symbolics.jl combination, mirroring what script 02 did for the Bernstein side.

Output explanation for `05-bezier-matrix-sympy-text.jl`, Case 2 | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated